

Stellar Level 1

lib.rs

```
#![no_std]
use soroban_sdk::{contract, contractimpl, contracttype, symbol_short, Env, String, Symbol,
Vec};

// Struktur data yang akan menyimpan notes
#[contracttype]
#[derive(Clone, Debug)]
pub struct Note {
    id: u64,
    title: String,
    content: String,
}

// Storage key untuk data notes
const NOTE_DATA: Symbol = symbol_short!("NOTE_DATA");

#[contract]
pub struct NotesContract;

#[contractimpl]
impl NotesContract {
    pub fn get_notes(env: Env) -> Vec<Note> {
        // 1. ambil data notes dari storage
        return env.storage().instance().get(&NOTE_DATA).unwrap_or(Vec::new(&env));
    }

    // Fungsi untuk membuat note baru
    pub fn create_note(env: Env, title: String, content: String) -> String {
        // 1. ambil data notes dari storage
        let mut notes: Vec<Note> =
env.storage().instance().get(&NOTE_DATA).unwrap_or(Vec::new(&env));

        // 2. Buat object note baru
        let note = Note {
            id: env.prng().gen:::<u64>(),
            title: title,
            content: content,
        };

        return String::from_str(&env, "Notes tidak ditemukan")
    }
}
```

```

}

mod test;

// 3. tambahkan note baru ke notes lama
notes.push_back(note);

// 4. simpan notes ke storage
env.storage().instance().set(&NOTE_DATA, &notes);

return String::from_str(&env, "Notes berhasil ditambahkan");
}

// Fungsi untuk menghapus notes berdasarkan id
pub fn delete_note(env: Env, id: u64) -> String {
    // 1. ambil data notes dari storage
    let mut notes: Vec<Note> =
env.storage().instance().get(&NOTE_DATA).unwrap_or(Vec::new(&env));

    // 2. cari index note yang akan dihapus menggunakan perulangan
    for i in 0..notes.len() {
        if notes.get(i).unwrap().id == id {
            notes.remove(i);

            env.storage().instance().set(&NOTE_DATA, &notes);
            return String::from_str(&env, "Berhasil hapus notes");
        }
    }
}

```

[test.rs](#)

```

#![cfg(test)]

use super::*;
use soroban_sdk::{Env, String};

// Melakukan test disini

```

Cargo.toml

```

[package]
name = "notes"
version = "0.0.0"
edition = "2021"
publish = false

```

```
[lib]
crate-type = ["lib", "cdylib"]
doctest = false
```

```
[dependencies]
soroban-sdk = { workspace = true }
```

```
[dev-dependencies]
soroban-sdk = { workspace = true, features = ["testutils"] }
```

Makefile

```
default: build
```

```
all: test
```

```
test: build
     cargo test
```

```
build:
     stellar contract build
     @ls -l target/wasm32v1-none/release/*.wasm
```

```
fmt:
     cargo fmt --all
```

```
clean:
     cargo clean
```

Cargo.toml

```
[workspace]
resolver = "2"
members = [
    "contracts/*",
]
```

```
[workspace.dependencies]
soroban-sdk = "25"
```

```
[profile.release]
opt-level = "z"
overflow-checks = true
debug = 0
strip = "symbols"
```

```
debug-assertions = false
panic = "abort"
codegen-units = 1
lto = true
```

```
# For more information about this profile see
https://soroban.stellar.org/docs/basic-tutorials/logging#cargotoml-profile
[profile.release-with-logs]
inherits = "release"
debug-assertions = true
```

[README.md](#)

```
.
├── contracts
│   └── hello_world
│       ├── src
│       │   ├── lib.rs
│       │   └── test.rs
│       └── Cargo.toml
├── Cargo.toml
└── README.md
```